

Documento de Solución

Servicio de Gestión de Tareas Orientado a Eventos — Opción A · Prueba técnica Ingeniero de Soluciones TI

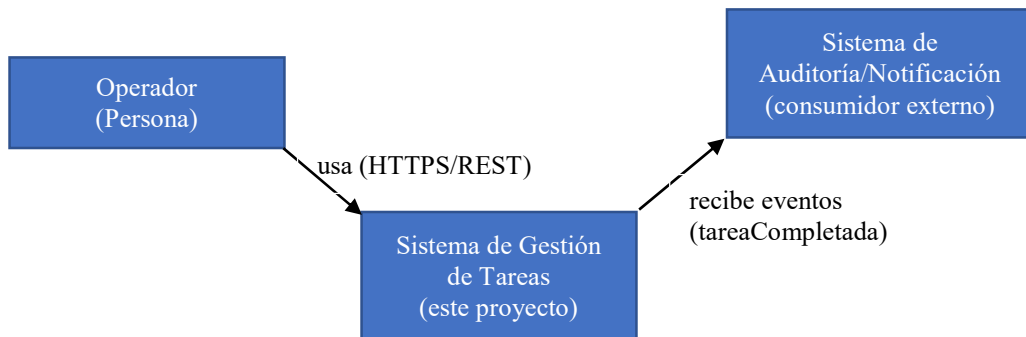
1. Contexto y alcance

Se construyó una API de gestión de tareas que permite crear, consultar, actualizar, completar y eliminar tareas, y que publica el evento de dominio **TareaCompletada** cuando una tarea cambia a estado completada. Un **worker independiente** consume ese evento y registra una traza de auditoría, demostrando desacoplamiento entre el servicio principal y sus consumidores.

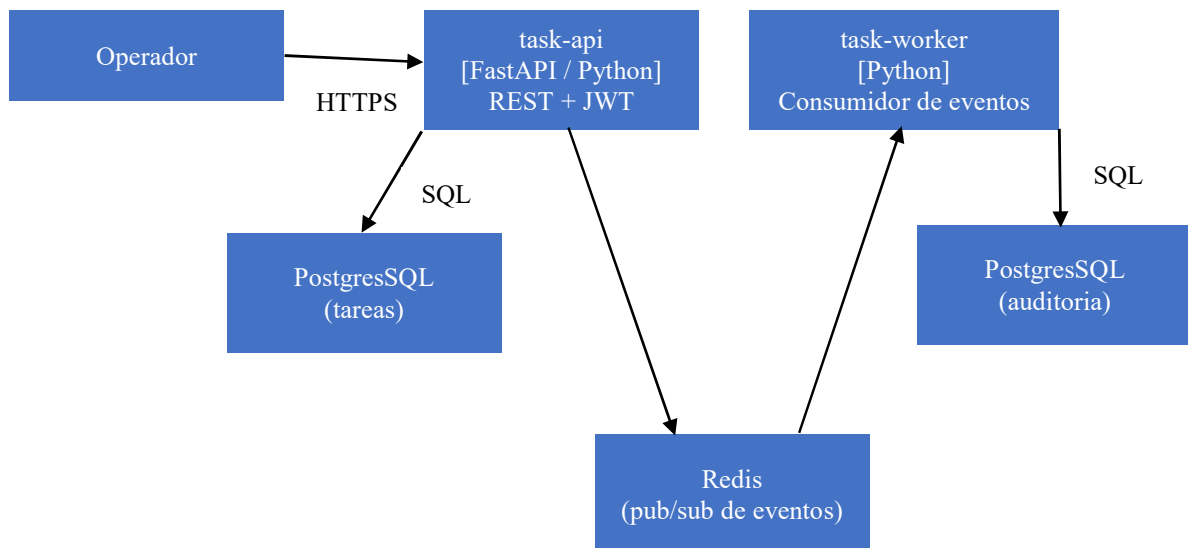
Fuera de alcance (y por qué): integración con un IdP real para autenticación (se usó JWT simplificado, ver ADR-003), notificaciones reales por correo/SMS (se simulan como registro de auditoría), interfaz gráfica (se prioriza calidad del backend y de la arquitectura de eventos, que es el énfasis de esta opción), y migraciones versionadas con Alembic (se usó create_all de SQLAlchemy por simplicidad en el tiempo disponible).

2. Arquitectura (modelo C4)

C4 – Nivel 1: Diagrama de contexto



C4 – Nivel 2: Diagrama de Contenedores



3. Registros de decisiones de arquitectura (ADR) — resumen

El detalle completo de cada ADR (contexto, alternativas y consecuencias) está en docs/adr/ del repositorio. Resumen:

ADR	Decisión	Alternativas descartadas
001	PostgreSQL para persistencia de tareas	MongoDB (esquema flexible innecesario); DynamoDB (dificulta filtros combinados)
002	Puerto EventPublisher con adaptador Redis pub/sub para la demo local	RabbitMQ (complejidad operativa innecesaria); SQS/SNS (requiere credenciales AWS para correr localmente)
003	JWT emitido por /auth/login (sin validar contraseña real)	API Key estática — sugerida inicialmente por la IA, descartada por no expresar expiración ni identidad (sub)

4. Requisitos no funcionales

Atributo	Cómo se aborda en el prototipo
Disponibilidad	Health check (/health); Docker Compose con healthchecks para DB y Redis antes de iniciar dependientes.
Seguridad	JWT en todos los endpoints de tareas; secretos vía variables de entorno (nunca en el repo); usuario no-root en contenedores; validación de entradas con Pydantic; SAST/SCA/detección de secretos en CI.
Escalabilidad	API sin estado (stateless) → escalable horizontalmente; worker desacoplado vía eventos, puede escalarse independientemente de la API; paginación en listados evita cargas grandes en memoria.
Observabilidad	Logs estructurados en API y worker; correlación básica por tarea_id en eventos; reporte de cobertura de pruebas en CI. Pendiente para producción: métricas (Prometheus/CloudWatch) y trazas distribuidas.

5. Riesgos técnicos identificados y mitigación

Riesgo	Mitigación propuesta
Pérdida de eventos: Redis pub/sub no persiste mensajes si el worker está caído.	Migrar a SNS+SQS con cola de mensajes fallidos (DLQ) en producción (ver sección 6).
Procesamiento duplicado de eventos (at-least-once delivery).	El worker implementa una deduplicación básica en memoria por task_id; en producción se persistiría el event_id procesado para garantizar idempotencia real entre reinicios.
Secretos filtrados accidentalmente en el repositorio.	GitLeaks en el pipeline de CI en cada push; .env excluido en .gitignore; .env.example sin valores reales.
Esquema de base de datos evoluciona sin control de versiones (se usa create_all).	Introducir Alembic con migraciones versionadas antes de operar en un ambiente compartido/productivo.
Token JWT sin mecanismo de revocación.	Vida corta del token (60 min) como mitigación temporal; en producción usar un IdP con revocación y rotación de llaves (JWKS).

6. Camino a producción (AWS, alta disponibilidad, entidad vigilada)

Arquitectura objetivo: API en contenedores sobre ECS Fargate (o EKS) detrás de un Application Load Balancer, en múltiples zonas de disponibilidad. Persistencia en Aurora PostgreSQL Multi-AZ. Eventos migrados de Redis pub/sub a Amazon SNS + SQS con DLQ, garantizando durabilidad y reintentos gestionados. El worker se despliega como un servicio ECS independiente con auto-scaling basado en profundidad de cola (backlog de SQS).

Seguridad: secretos en AWS Secrets Manager (nunca variables de entorno planas); autenticación delegada a Amazon Cognito o el IdP corporativo, con JWT validado vía JWKS; cifrado en tránsito (TLS 1.2+) y en reposo (KMS) para base de datos y colas; WAF frente al ALB; cumplimiento con lineamientos de la Superintendencia Financiera sobre continuidad y trazabilidad de operaciones (registro de auditoría inmutable, retención documentada).

Monitoreo y continuidad: métricas y alarmas en CloudWatch (latencia, tasa de error, profundidad de DLQ), trazas distribuidas con AWS X-Ray/OpenTelemetry, dashboards de salud por servicio. Estrategia de continuidad: backups automáticos de Aurora con recuperación point-in-time, despliegue Multi-AZ, y un plan de recuperación ante desastres (RTO/RPO definidos según criticidad del proceso) con réplica en una segunda región si el proceso se clasifica como crítico para la entidad.

CI/CD: pipeline con despliegue continuo a un ambiente de staging tras pasar pruebas y controles de seguridad (SAST/SCA/secret scanning ya presentes en el pipeline actual), con aprobación manual para producción e infraestructura como código (Terraform) versionada junto al aplicativo.